

G1 Branch Strategy & The Case for the New Stack

2026-07-31 · rev. 2 · For Adrián · Which code goes on the REAL robot for which test, exact checkout commands, and what the tutor-feedback stack buys you — every number is measured, nothing is promised.

1 The complete ref inventory (what exists and what each one contains)

Ref (exact name)	Type / where	What it contains
golden-doorvis	tag (= commit 019ec85), on GitHub	The FROZEN 24-jul golden baseline: DOOR-VIS, the code that produced 5 consecutive clean real runs (B→A 52–60 s, 0 collisions). Never moves.
main	branch, on GitHub	Mainline development: RETREAT v2, COLOR-BRAKE, DOORSTICKY v3, ENV-CHANGE shadow, perception fixes, REALCAL, config snapshots, goto.log auto-rotation. Does NOT have the meta state machine.
tutor-feedback-metareasoner	branch, LOCAL Mac tip ahead of GitHub (push held)	golden + Renxi's meta-spec batches 1+2 (human channel, laser_trust, door_contra, iface_q, META state machine, RETREAT v2 ported, RETREAT-AT-EXIT) + twin campaigns + sensor noise + synthetic camera. Push waits for the goto.log/GPUEDGE coordination.
tutor-feedback-metareasoner-sim	branch, on GitHub (the published copy)	Identical content to the row above plus the docs (twin test instructions, R1 checklist, operator runbook, this document). Use this name.

See them all at any time:

```
cd "/Users/adrianlendinezibanez/Claude/Projects/G1 R0BOT" && git fetch origin && git branch -a &
& git tag
```

2 Which ref for which REAL test — with the exact commands

Real test	Ref to use	Checkout command (from the repo folder)
R1 re-baseline (next session, 10 runs, Gate 1)	golden-doorvis	git checkout golden-doorvis (this leaves you in "detached HEAD" — normal and fine for a session; do not commit code here)
R2 water 200 g GOV/BASE ABBA	main	git checkout main
R3a FULLCUP 247 g	main	git checkout main
R3b main-vs-golden promotion	BOTH, alternating per arm	git checkout main for the main arm · git checkout golden-doorvis for the golden arm
R4a wrong-prior ×6 · R4b sealed lid ×4	main	git checkout main
Real debut of the NEW stack (meta state machine + human keys; after Gate 1)	tutor-feedback-metareasoner-sim	git fetch origin && git checkout tutor-feedback-metareasoner-sim

After ANY session or experiment, leave the tree back on main (house convention):

```
cd "/Users/adrianlendinezibanez/Claude/Projects/G1 R0BOT" && git checkout main
```

Quick sanity check that you are on the right code before the first run:

```
git log --oneline -1 && git status --short | head -5
```

Expected: R1 → the line shows 019ec85 ; new-stack session → tip ≥ 0480988 ; status shows no modified source files.

Mental rule: golden-doorvis = MEASURE · main = THE PLAN (R2–R4) · tutor-feedback-metareasoner-sim = THE NEW WORK. Nothing new touches the robot before a fresh re-baseline exists to compare it against. (R2–R4 assignments follow *G1_Results_Acquisition_Plan.pdf* — if that document and this table ever disagree, the plan wins.)

Run command for the new-stack real session (machine flags are default-ON on the branch)

```
cd "/Users/adrianlendinezibanez/Claude/Projects/G1 ROBOT" && git fetch origin && git checkout tutor-feedback-metareasoner-sim && G1_META2=2 G1_M2_CFG=config_meta2_g1payload.json G1_M2_STATE=meta2_state_branch_real.json G1_M2_STATE_INIT=meta2_state_twin_explored.json G1_M2_REALCAL=1 G1_M2_L3=1 G1_M2_L4=1 G1_M2_PROFILES=1 G1_M2_D00RLIB=1 G1_M2_FRAGSPEED=1 G1_M2_ABORT_WIN=150 G1_M2_ABORT_PROG=0.25 G1_M2_HELP_S=16 G1_D00R_VIS=1 G1_PERC=192.168.51.80:8008 /Users/adrianlendinezibanez/unitree_webrtc_connect/.venv/bin/python g1_goto.py goto B
```

Marker v3 in a second terminal (`python3 spill_mark.py`): ENTER=spill · w <g>=weigh · i=invalid · **c=collision the robot missed** · m <cm>=**you moved/helped the robot**. The c / m keys only exist on this branch — on golden/main the marker is v2 (no c/m).

3 What the new stack buys you (measured)

#	Advantage	The number behind it
1	Prevention under degraded sensing. Conservative caps in blind/noisy zones stop wedges from forming at all.	Closed loop under calibrated noise: OFF 3/6 reached, 27 collisions — ON 6/6, 0 collisions
2	Near-zero-cost insurance when things are fine. It does not touch good runs.	0 false interventions in clean closed loop; golden-window replay 10/10 untouched; 2/192 (1.0%) false positives over all clean real runs. Price: +8.6% time (median 93→101 s)
3	Structured recovery instead of grinding. Breadcrumb retreat along the known-good corridor.	Real data: 58% of stuck episodes NEVER self-freed; those that did took median 26 s (p75 52 s). The retreat undoes 0.7–0.8 m in ~14 s; 15/18 executed retreats un-wedged the robot (c0 0.3 → >0.75 m)
4	Human-in-the-loop rescue. After 3 failed retreats the robot STOPS and asks; your <code>m <cm></code> re-arms its budgets and it resumes.	Demonstrated twice in one twin run. Retrospective over your real history: 35 runs would have asked for help; 48 minutes of post-request wrestling were cuttable. In real sessions a human is always present
5	Laser lie detection. The robot finally knows when its LiDAR deceived it — and you can tell it (<code>c</code>) about hits it missed.	55% of all real collisions (107/193) happened with the laser reading clear >0.6 m
6	Start-pocket protection. Retreats can now arm right at the waypoint exit (the "B nose-to-sofa" geometry).	Wedge draws: 1–2 collisions with structured close vs 7–14 grinding before; retreats armed at 0.40–0.47 m from start (previously impossible)
7	Evidence-grade observability. Every run logs <code>meta_state</code> , <code>iface_q</code> , <code>laser_trust</code> , <code>door_contra</code> + transition events.	Renxi's meta-spec implemented 1:1; every run analysable with <code>analyze_msm.py</code> / replayable with <code>replay_msm.py</code> — thesis-ready evidence for Ch.5

4 Honest caveats (say them before Renxi does)

- **Twin-validated, not real-validated.** All closed-loop numbers come from the digital twin under noise calibrated against your real 24-jul distributions. The real trial is exactly what the post-Gate-1 experiment is for.
- **+8–10% time in clean conditions** — the price of the BLIND caps. On golden-form days it is insurance you rarely need; on bad-sensing days it is the difference between 6/6 and 3/6.
- **ASSIST does not latch** (it can drop out if clearance rises during an align dance). Behaved correctly in every test; latched semantics is a known v2 item.
- **DEGRADED needs ≥3 missed-collision reports in ~30 s** — a deliberately high bar; you will rarely see it, but sub-threshold trust dips are logged anyway.
- **DOOR-VIS was a tie in the twin** (4/6 vs 4/6, same door times): the twin's door is not the bottleneck; the golden benefit came from real map-reality mismatch. Keep it ON on the real robot (zero measured cost).
- **The B-pocket root cause is physical** (waypoint recorded too close to furniture) — the new stack manages it; re-recording the waypoint or censusing the obstacle would remove it.

5 Suggested deployment sequence

1. **Session 1 — R1** on `golden-doorvis`: 10 runs, Gate 1. (Checklist: `docs/G1_R1_Session_Checklist.pdf`.)

2. **Gate 1 PASS** → continue the plan (R2 on `main` ...) *or* insert the new-stack trial next if you and Renxi prefer capability evidence first.
3. **New-stack session** on `tutor-feedback-metareasoner-sim`: repeat the SAME 10-run pattern as R1 (5+5 alternating, 247 g, weigh every leg) so the comparison is 1:1 against your fresh baseline. Practise the two new keys: `c` when you see a hit the robot missed, `m <cm>` after you physically help it during ASSIST.
4. **Compare** (reached %, collisions, spills/leg, time; then `analyze_msm.py` per run) and decide the merge with data on the table.

One-line answer to "which branch?": R1 on `git checkout golden-doorvis` · the plan's tests on `git checkout main` (R3b alternates both) · the new stack's real debut on `git checkout tutor-feedback-metareasoner-sim`, after Gate 1, mirrored 1:1 against the fresh baseline · always back to `git checkout main` when you finish.